

02/10/25

Test Driven Development → Desarrollo conducido por pruebas

No es una metodología de prueba, es una forma de construir código.

P.V.D. — Requisitos

→ Analisis

→ Diseño

→ Implementación

→ Pruebas / Testing → 30% - 50% del costo total del proyecto

→ Despliegue

↓
no significa q la gente
haga esto

Antes hay que diseñar los casos de prueba → artefacto + importante que sale del testing

↓
un caso de prueba es
determinar si uno o más criterios de aceptación pasan o no

Precondiciones → conj. de situaciones q deben ser válidas

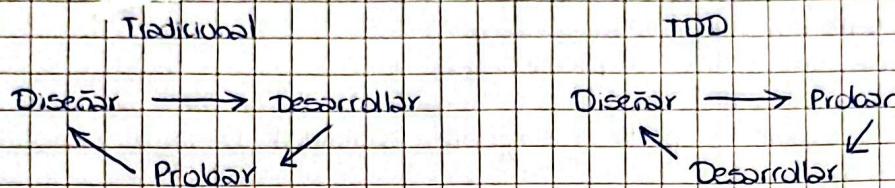
Tomo los req., diseño los casos de prueba.

El problema del tiempo es q solo estiman el tiempo de programación.

Alguien propone adelantar el Testing, adelantar todo lo q se pueda antes del código.

Es muy importante ir pensando el sw q vamos a construir.

La confirmación está materializada en los casos de prueba.



Hay una concepción de q la gente de Testing viene a destruir.

Hay una subestimación importante respecto del trabajo de Testing.

Calidad → adecuarse a los req. q el cliente pidió
es una decisión

La gestión de configuración es un problema de todos.

Personas de operación o de soporte.

DevOps → Development + Operación
el código q vos construís, vos te hacés cargo para que siga funcionando.

Agilismo → impulsa la idea de que se puede funcionar sin detalles en los req., si no se tomaron decisiones el desarrollador toma la decisión.
poner el detalle donde hace falta y donde son imprescindibles → en las pruebas los req. y US no son tan detalladas

Gestión tradicional → los req. eran detallados y los casos de prueba + todavía.

En gual, muchos desarrolladores programan y le piden al tester q le diga donde corregir. I porta la actitud.

Cuatro niveles de pruebas

Pruebas unitarias → la hace el propio desarrollador
única q se hace en la implementación

Pruebas de integración → querés probar si cumple con todos (F y NF) los requerimientos.

Sistema o versión

UAT - Pruebas de aceptación de usuario → la tiene que hacer el usuario
se hacen en el despliegue

PUD → iterativo e incremental → mini cascada en cada iteración

TDD → técnica, primero escribimos CP, estos habilitan la construcción del componente, luego se ejecutan las pruebas.
está pensado a nivel de pruebas de unidad

Extreme Programming (XP) → Framework ágil
TDD empezó siendo una actividad de acá

TDD → evolución → ATDD
BDD → desarrollo conducido por comportamiento

El primer test falla, chequeo el código, refactorizo → es un ciclo

Leves → no escribir una línea de código hasta q no se haya escrito un test case
con un test q falle es suficiente
no vas a escribir más código en producción.

Refactoring → forma de desarrollar q apunta a mejorar la calidad de un componente q desarrollamos
no cambiamos la funcionalidad, queremos mejorar el nivel de cohesión y acoplamiento
suma a TDD, para q cuando corregimos un componente para q pase la prueba, usamos refactoring

Cada error q se corrige introduce 3.

Práctico de TDD

Precondiciones o setup: estado inicial necesario para ejecutar correctamente los casos de prueba.

Nombre: genérico → def test_nombre_us_caminos_resultado()
↓
success/fail

Asociar los CP a ejemplos

Escenario → taxis libres (identificador verde) con chapa válida para un barrio existente.